

Unit 2: Python Control Flow, Functions & Modules • Topic 6

Python Modules (import, Standard Library, pip)

Organising and reusing code with modules: the import statement, the standard library, and installing packages with pip.

As programs grow, it becomes impractical to write everything from scratch or keep all code in one file. A **module** is simply a file containing Python code (functions, variables, classes) that can be reused across programs. Python's true strength comes from its enormous collection of ready-made modules — both built in and installable.

6.1 What is a Module?

A **module** is any Python file (.py) whose functions and variables can be reused in another program by **importing** it. A module lets you organise related code together, avoid rewriting the same logic repeatedly, and reuse code written by others (the Python community has published thousands of modules).

6.2 The import Statement

Python provides several ways to bring a module's contents into your program:

(a) import module_name

Imports the whole module; its functions/variables must be accessed using module_name.item.

```
import math
print(math.sqrt(25))      # 5.0
print(math.pi)            # 3.141592653589793
```

(b) import module_name as alias

Imports a module under a shorter or clearer name, commonly used for modules with long or frequently used names.

```
import math as m
print(m.factorial(5))     # 120
```

(c) from module_name import item

Imports only specific function(s)/variable(s) directly, so they can be used without the module-name prefix.

```
from math import sqrt, pi
print(sqrt(16))           # 4.0
print(pi)                 # 3.141592653589793
```

(d) from module_name import *

Imports **everything** from a module directly into the current namespace. This is generally **discouraged** in real programs because it can silently overwrite existing names and makes it unclear where a function came from.

```
from math import *
print(sqrt(9), pi, factorial(4))  # works, but avoid this style in larger programs
```

6.3 The Python Standard Library

Python comes bundled with a huge collection of ready-to-use modules called the **standard library** — no installation needed, just import. Some of the most commonly used ones:

Module	Purpose	Example
math	Mathematical functions and constants	math.sqrt(x), math.pi, math.factorial(n)
random	Generating random numbers/choices	random.randint(1,6), random.choice(list)
datetime	Working with dates and times	datetime.date.today()
os	Interacting with the operating system	os.getcwd(), os.listdir()
sys	Interacting with the Python interpreter itself	sys.exit(), sys.argv
statistics	Statistical calculations	statistics.mean([1,2,3])

```
import random
dice_roll = random.randint(1, 6)      # random integer between 1 and 6, inclusive
print("You rolled:", dice_roll)

import datetime
today = datetime.date.today()
print("Today's date is:", today)
```

6.4 pip — The Python Package Installer

Beyond the standard library, the Python community publishes third-party packages on the **Python Package Index (PyPI)** — e.g. numpy, pandas, matplotlib (all used later in this course). **pip** is Python's official package manager, used from the command line/terminal (not inside a Python script) to install, upgrade, and remove such packages.

```
# Run these commands in a terminal / command prompt, NOT inside a .py file:

pip install numpy           # installs the latest version of numpy
pip install pandas==2.1.0   # installs a specific version
pip uninstall numpy         # removes a package
pip list                    # shows all installed packages
pip show numpy              # shows details about an installed package
```

Once installed via pip, a package is imported exactly like any other module:

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr)
```

GOOD PRACTICE: A project's exact package requirements are often listed in a requirements.txt file, so anyone can recreate the same environment with a single command: `pip install -r requirements.txt`.

6.5 Creating and Using Your Own Module

Any Python file you write can itself be imported as a module by another file in the same folder — there is nothing special to declare.

```
# File: calc_utils.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# File: main.py (in the same folder)
import calc_utils
print(calc_utils.add(5, 3))      # 8
print(calc_utils.subtract(5, 3)) # 2
```

6.6 import vs from-import vs pip — Quick Recap

Concept	Purpose	Used inside a .py file?
import module	Bring in a whole module, used with a prefix	Yes
from module import x	Bring in specific names directly, no prefix needed	Yes
pip install package	Download and install a third-party package onto the system	No — run in a terminal

Practice Questions

A. Multiple Choice Questions

- Which statement correctly imports only the `sqrt` function from the `math` module?
(A) `import sqrt` (B) `from math import sqrt` (C) `import math.sqrt` (D) `using sqrt from math`
- Where should the command '`pip install numpy`' be typed?
(A) Inside a Python script using `print()` (B) In the terminal/command prompt (C) Inside a for loop (D) As a comment in the code
- Which module would you use to generate a random number in Python?
(A) `math` (B) `os` (C) `random` (D) `sys`
- What does '`import math as m`' do?
(A) Imports only the variable `m` from `math` (B) Imports the `math` module under the alias `m` (C) Causes a `SyntaxError` (D) Installs the `math` package
- Why is '`from module import *`' generally discouraged?
(A) It is invalid syntax (B) It can overwrite existing names and hides where functions came from (C) It only works with the `math` module (D) It requires `pip`

B. True / False

- The `math` module is part of Python's standard library and needs no installation.
- `pip` is used inside a Python program to import a module.
- A Python file you write yourself can also be imported as a module.
- `from math import pi` allows you to use `pi` directly without the `math.` prefix.

C. Short Answer Questions

- Differentiate between the standard library and third-party packages installed via `pip`.
- List and explain the four different forms of the `import` statement, with one example each.
- What is `pip`, and what is the purpose of a `requirements.txt` file?
- Why might a programmer prefer '`import math`' over '`from math import *`' in a large project?

D. Long Answer / Descriptive Questions

- Explain how to create your own Python module and import it into another file, with a complete example containing at least two functions.
- Describe five commonly used modules from Python's standard library, the purpose of each, and give one example function/use from each module.

E. Programming Problems

- Write a Python program that uses the `random` module to simulate rolling two dice 5 times and prints each roll's total.
- Write a Python program that uses the `math` module to calculate the area of a circle ($\text{area} = \pi * r^2$) given its radius, and prints the result rounded to 2 decimal places.
- Create your own module named `string_utils.py` containing a function `count_vowels(text)` that counts vowels in a string, then write a second file that imports and uses it.

19. Write a Python program that uses the datetime module to print today's date and the day of the week.

Answer Key (A & B): MCQ: 1-B, 2-B, 3-C, 4-B, 5-B | T/F: 6-True, 7-False, 8-True, 9-True